

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	AMARA SATWIKA	Reg.No.:	192525204
Department:	AIML	Date:	26.08.26

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

Real-Time Embedded System I/O Communication Mechanism

Problem Understanding

The system has multiple sensors generating interrupts every 1 ms. Therefore, the I/O mechanism must provide very low latency while keeping CPU utilization below 15%.

Proposed Architecture

Sensors → Interrupt Controller → CPU → Memory

- Use a vectored interrupt controller.
- Each sensor is assigned a separate interrupt vector.
- When a sensor generates an interrupt, the CPU directly jumps to its corresponding Interrupt Service Routine (ISR).
- Use memory-mapped I/O for accessing sensor registers.
- Use short ISRs to reduce CPU utilization.
- For larger sensor data, DMA can be used so that the CPU does not continuously transfer data.

Feature	Memory-Mapped I/O	I/O-Mapped I/O
Address space	Uses memory address space	Separate I/O address space
Instructions	Normal memory instructions	Special I/O instructions
Access	Fast and flexible	More restricted
Suitable here	Yes	Less suitable

Justification

Memory-mapped I/O is selected because it provides simple and fast register access, which is important for the 1 ms interrupt interval. Vectored interrupts reduce interrupt-processing time because the CPU can immediately identify the required ISR.

Final architecture:

Sensors → Vectored Interrupt Controller → CPU → Memory-Mapped Sensor Registers

This satisfies the <15% CPU utilization and low-latency requirements.

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

High-Speed Storage I/O Subsystem

Problem Understanding

The storage system requires throughput of more than 3 GB/s, must use NVMe over PCIe, and the CPU should not perform bulk data transfers.

Proposed Architecture

- NVMe SSD → PCIe → DMA Controller → Main Memory
- NVMe SSD → Interrupt → CPU

Working

1. The CPU sends an I/O request to the NVMe controller.
2. NVMe communicates with the system through PCIe.
3. The DMA controller transfers data directly between the NVMe device and main memory.
4. The CPU is not involved in every byte/word transfer.
5. After completing the transfer, the device generates an interrupt.
6. The CPU handles the completion notification.

Why NVMe + PCIe?

- PCIe provides high-bandwidth communication.
- NVMe is designed specifically for high-speed solid-state storage.
- DMA minimizes CPU involvement.
- Interrupt notification informs the CPU when the operation is complete.

Justification

DMA is essential because transferring more than 3 GB/s using CPU-controlled I/O would consume significant CPU resources. PCIe + NVMe + DMA provides high throughput while keeping CPU overhead low.

Final architecture:

- CPU → NVMe Controller → PCIe → DMA → Main Memory
- NVMe Controller → Interrupt → CPU
- This satisfies all the given constraints.

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

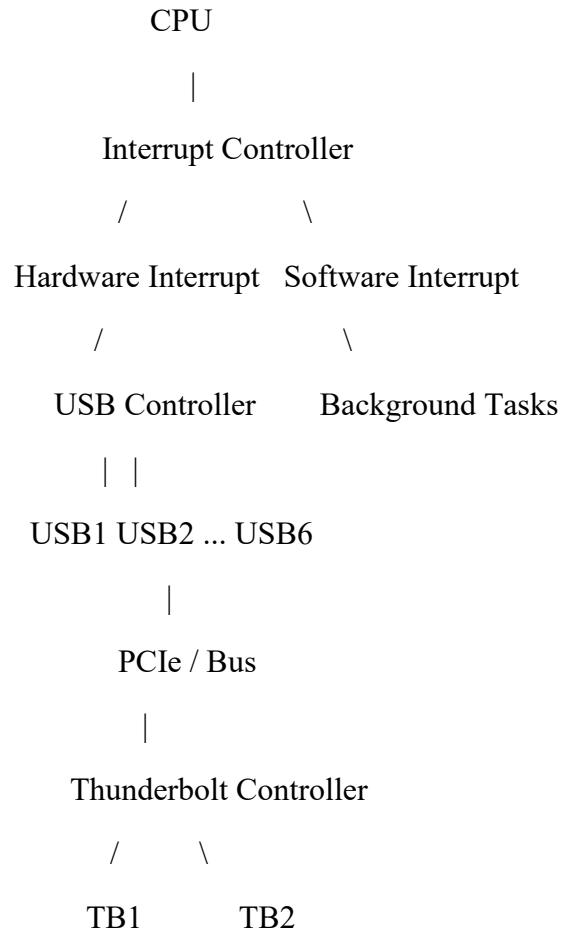
Multi-Device Communication Architecture

Problem Understanding

The system contains:

- 6 USB peripherals
- 2 high-speed Thunderbolt devices
- Hardware and software interrupts
- No interrupt starvation
- Fair bus arbitration

Proposed Architecture



Interrupt Handling

Hardware interrupts are used for time-critical device events such as:

- USB data arrival
- Thunderbolt transfer completion
- Device errors

Software interrupts are used for:

- Background processing
- Operating-system services
- Non-critical communication tasks

Avoiding Interrupt Starvation

Use:

- Interrupt priorities
- Interrupt masking carefully
- Round-robin servicing for devices with similar priority
- Short ISRs

High-priority interrupts are serviced first, but lower-priority devices must also receive CPU service.

Fair Bus Arbitration

A round-robin arbitration mechanism can be used.

Example:

USB1 → USB2 → USB3 → ... → USB6 → TB1 → TB2 → repeat

Each device receives an opportunity to use the bus, preventing one device from continuously occupying it.

Justification

Round-robin arbitration provides fairness, while interrupt prioritization ensures that critical events receive fast service. Hardware interrupts handle real-time events, while software interrupts handle background operations.

Therefore, the architecture satisfies the requirements for 8 peripherals, interrupt fairness, and bus fairness.

4.Design a data acquisition system with constraints:

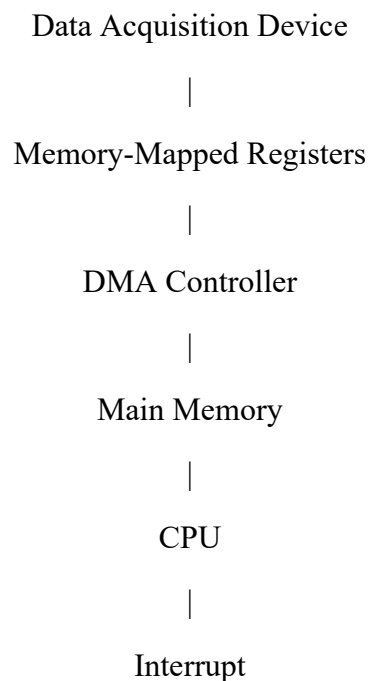
- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

Data Acquisition System

Problem Understanding

The system continuously receives data at 500 MB/s. CPU intervention must be minimal. Device registers must use memory-mapped I/O, and DMA must be compared with interrupt-driven I/O.

Proposed Architecture



DMA Transfer

1. CPU configures the DMA controller.
2. DMA receives data from the acquisition device.
3. DMA directly transfers the data to main memory.
4. CPU does not handle every data item.
5. DMA generates an interrupt after completing a block transfer.

DMA vs Interrupt-Driven I/O

Feature	DMA	Interrupt-Driven I/O
CPU involvement	Very low	High
Suitable for 500 MB/s	Yes	No
Data transfer	Device ↔ Memory	Device ↔ CPU ↔ Memory
CPU overhead	Low	High
Efficiency	High	Lower
Best use	Large continuous data	Small/moderate data

Justification

At 500 MB/s, interrupting the CPU for every data transfer would create excessive interrupt overhead. DMA transfers large blocks directly into memory and only interrupts the CPU after a block is completed.

Therefore, DMA is the preferred solution.

Final architecture:

Device → Memory-Mapped Registers → DMA → Main Memory → CPU

This provides continuous high-speed acquisition with minimal CPU intervention.

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency $< 10 \mu s$
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

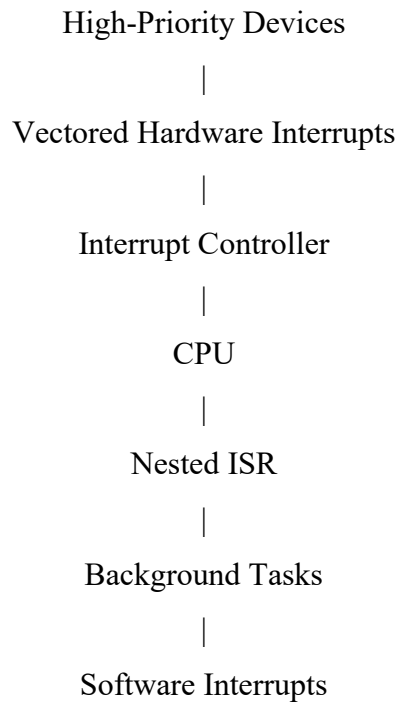
Interrupt Handling Mechanism

Problem Understanding

The interrupt system must:

- Support nested interrupts
- Maintain maximum interrupt latency below $10 \mu s$
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Proposed Architecture :



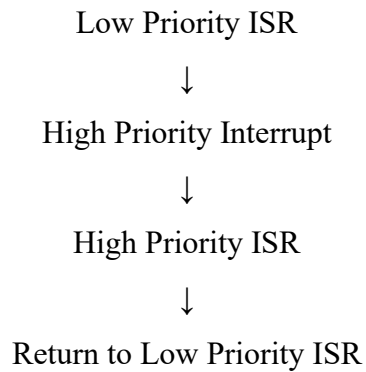
Working :

1. A high-priority device generates a hardware interrupt.
2. The interrupt controller identifies its vector.
3. CPU immediately jumps to the corresponding ISR.
4. If another higher-priority interrupt occurs, the current ISR can be temporarily interrupted.
5. The higher-priority ISR executes first.
6. After completion, the CPU returns to the previous ISR.
7. Software interrupts are used for background/non-critical tasks.

Nested Interrupts :

Nested interrupts allow a higher-priority interrupt to interrupt a lower-priority ISR.

Example:



Reducing Interrupt Latency :

To maintain $<10\ \mu\text{s}$ latency:

- Use vectored interrupts.
- Keep ISRs short.
- Use hardware priority levels.
- Avoid unnecessary interrupt masking.
- Save and restore only required CPU registers.
- Use efficient interrupt-controller hardware.

Justification :

Vectored interrupts provide fast identification of the interrupt source. Nested interrupts ensure that critical devices are serviced immediately. Software interrupts keep background tasks separate from urgent hardware events.

Therefore, the proposed mechanism satisfies the nested interrupt, $<10\ \mu\text{s}$ latency, vectored interrupt, and software interrupt requirements.

Code of Conduct:

I **A.SATWIKA / 192525204**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A.SATWIKA

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided